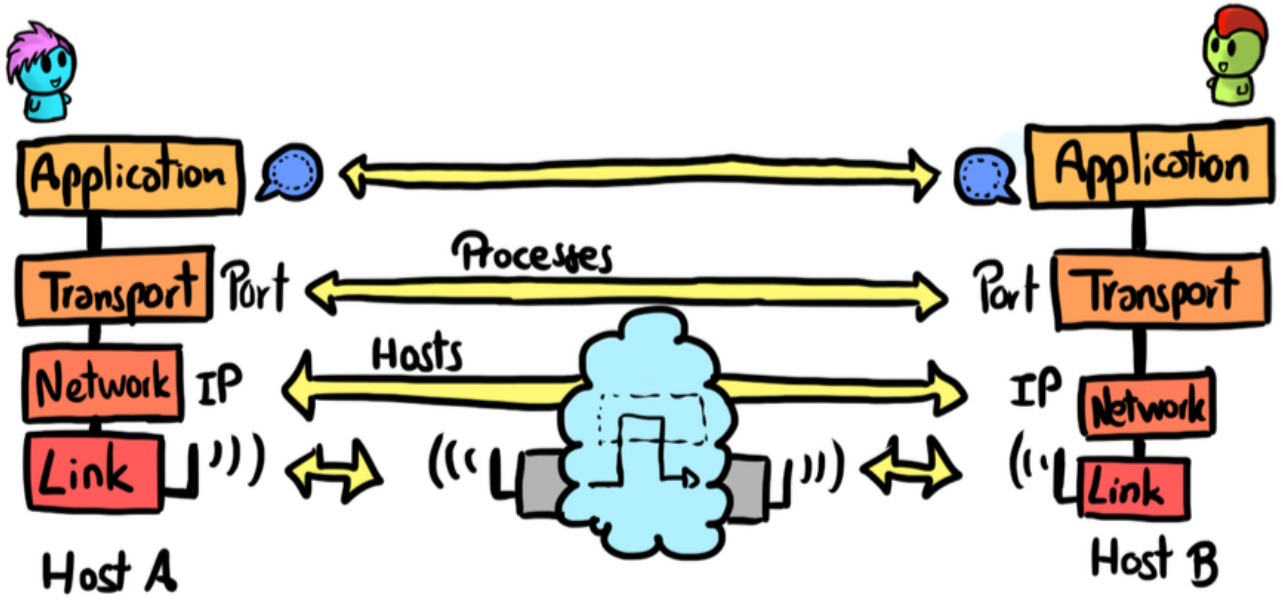
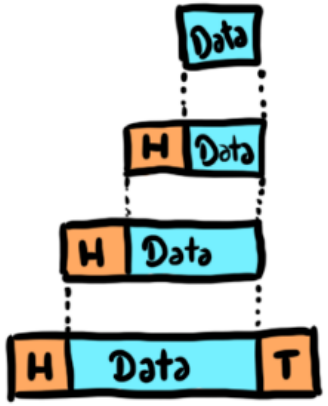
CS459/659 Privacy, Cryptography, Network and Data Security

Security through the layers

Spring 2026, Tuesday/Thursday 10:00-11:20am

Recall, the Network Stack



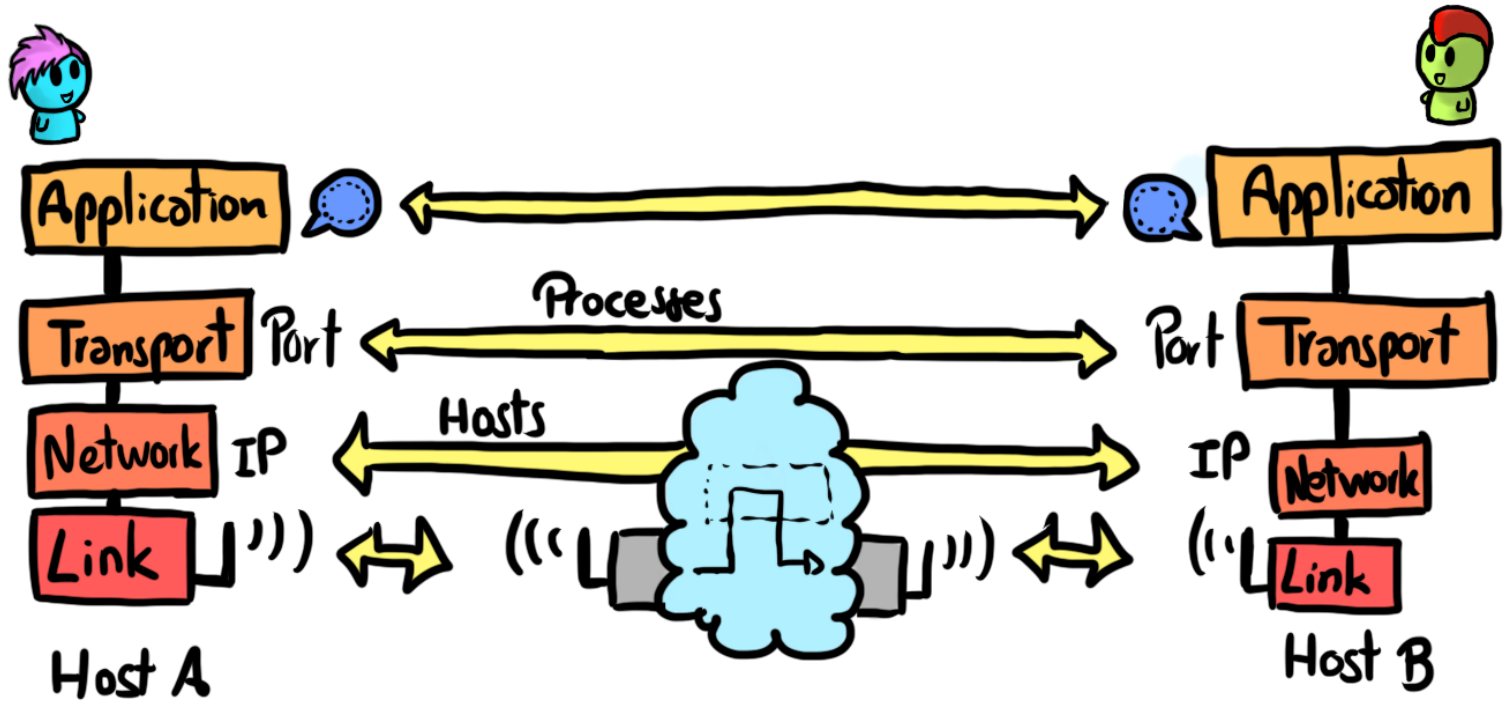


Q: Where do we need to apply crypto? (confidentiality, integrity, authentication)

- A Link layer is enough
- B Application layer is enough
- C We need it in all layers
- D Who needs crypto?

Today's Lecture – Security through the layers

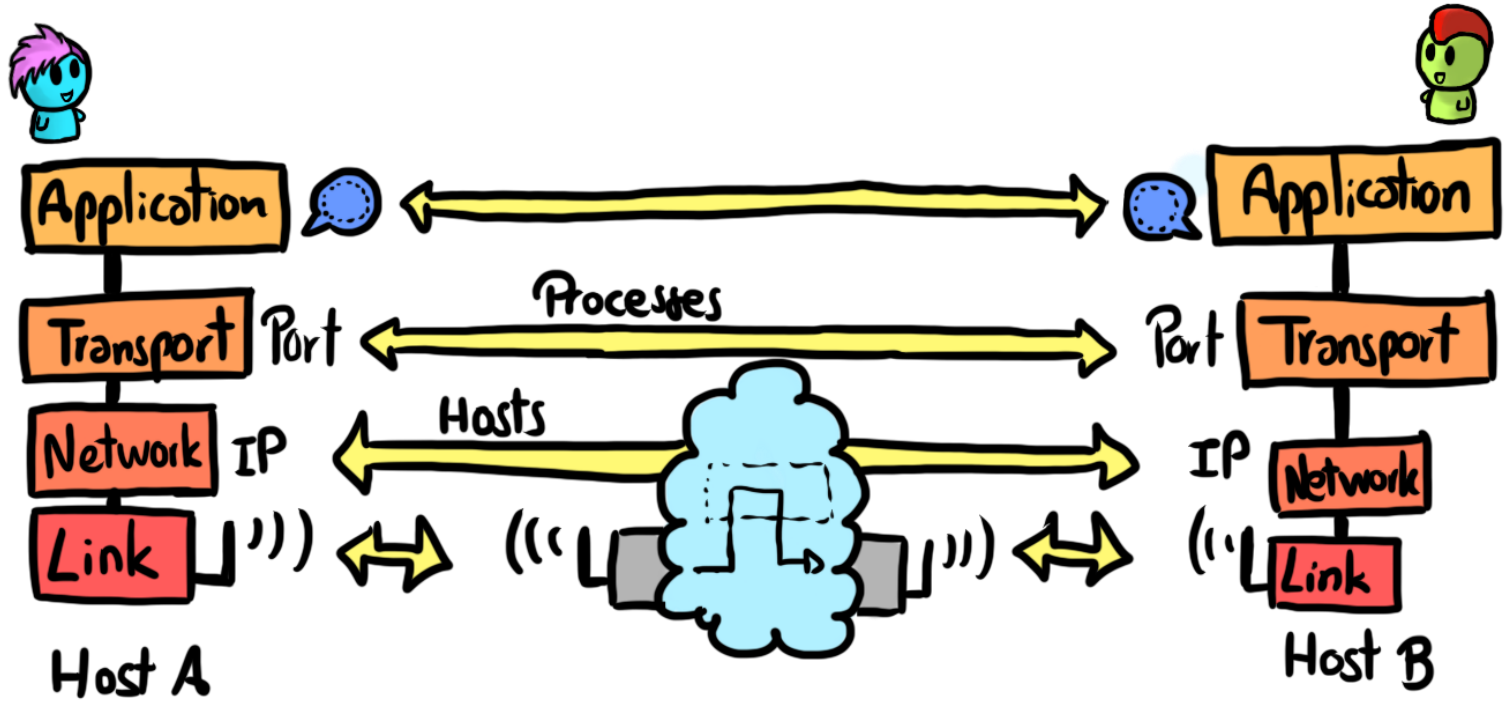
- Link
  - WEP, WPA, WPA2
- Network
  - VPN, IPsec
- Transport
  - TLS/SSL
- Application
  - ssh (Next class: PGP, OTR, Signal)



Link Layer – WPA2

Security through the layers

- Link
  - WEP, WPA, WPA2
- Network
  - VPN, IPsec
- Transport
  - TLS/SSL
- Application
  - ssh (Next class: PGP, OTR, Signal)



The history of Wi-Fi Security

- WEP - Learn From Mistakes
  - 1999
- WPA - Temporary Patch
  - 2003
- WPA2 - Mostly Ok
  - 2004
- WPA3 – Current Standard
  - 2018



Wired Equivalent Privacy (WEP)

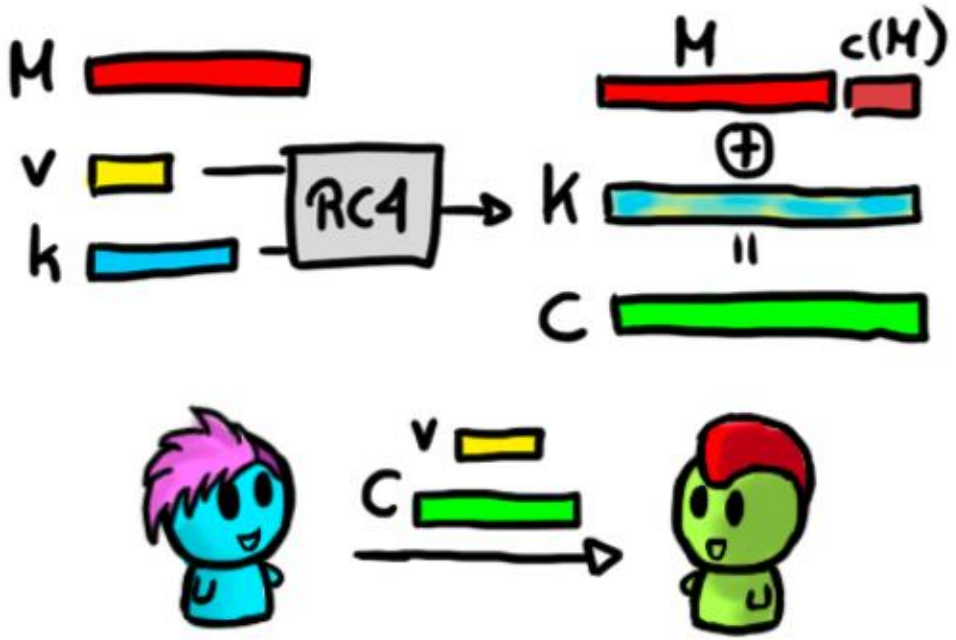
WEP was intended to enforce three security goals:

- Data Confidentiality
  - Prevent an adversary from learning the contents of the wireless traffic
- Data Integrity
  - Prevent an adversary from modifying the wireless traffic or fabricating traffic that looks legitimate
- Access Control
  - Prevent an adversary from using your wireless infrastructure

Unfortunately, none of these are actually enforced!

WEP Protocol

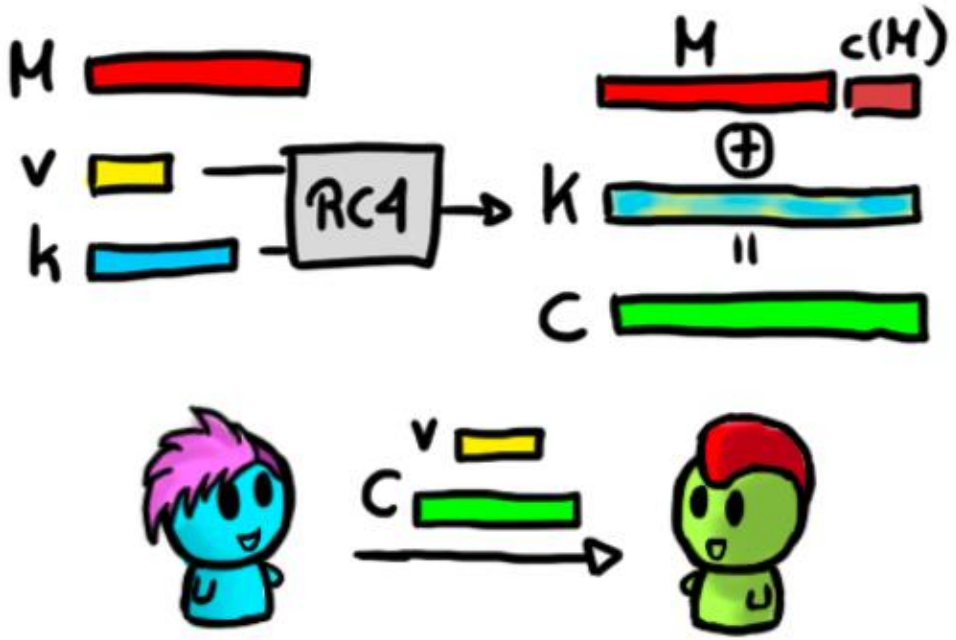
- The sender and receiver share a secret k (either 40 or 104 bits)
- In order to transmit a message M:
  - Compute a checksum $c(M)$ (which does not depend on k)
  - Pick an IV v and generate a keystream $K = RC4(v, k)$
  - Ciphertext $C = K \oplus \langle M \parallel c(M) \rangle$
  - Transmit v and C over the wireless link



Q: What kind of cipher is this?

WEP Protocol

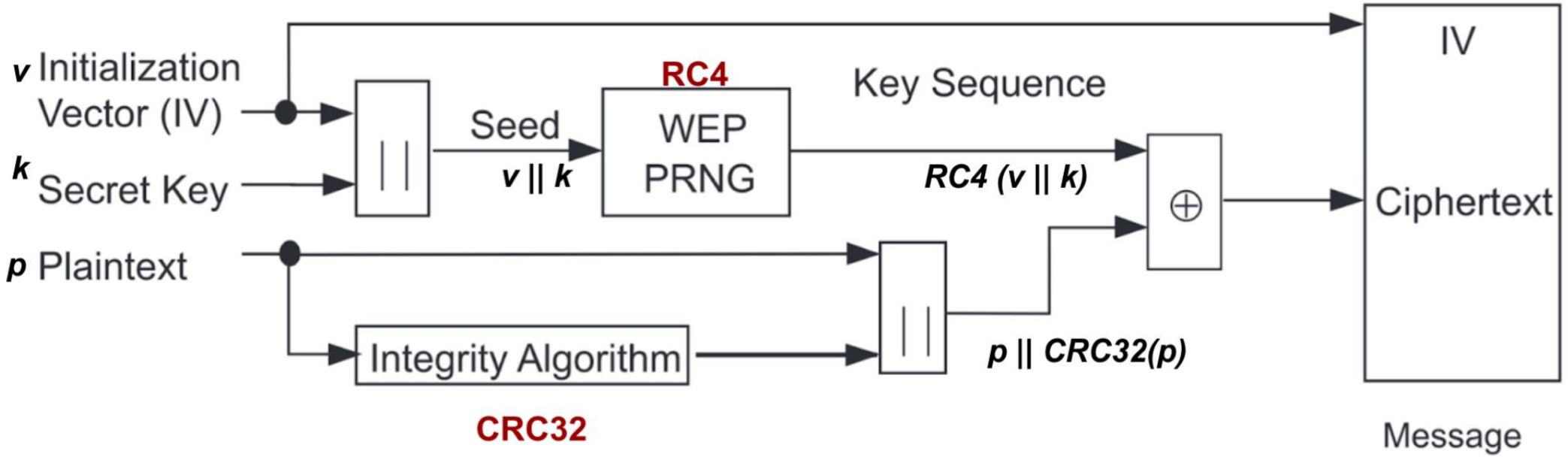
- The sender and receiver share a secret k (either 40 or 104 bits)
- In order to transmit a message M:
  - Compute a checksum $c(M)$ (which does not depend on k)
  - Pick an IV v and generate a keystream $K = RC4(v, k)$
  - Ciphertext $C = K \oplus \langle M \parallel c(M) \rangle$
  - Transmit v and C over the wireless link



Q: What kind of cipher is this?

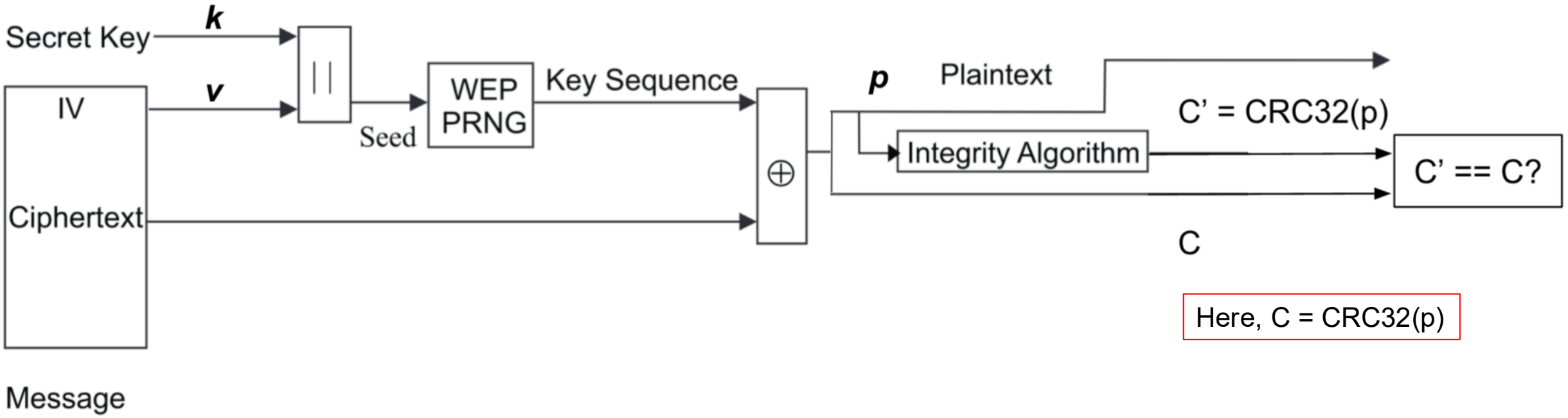
A: It's a stream cipher (symmetric)

WEP Encryption Algorithm



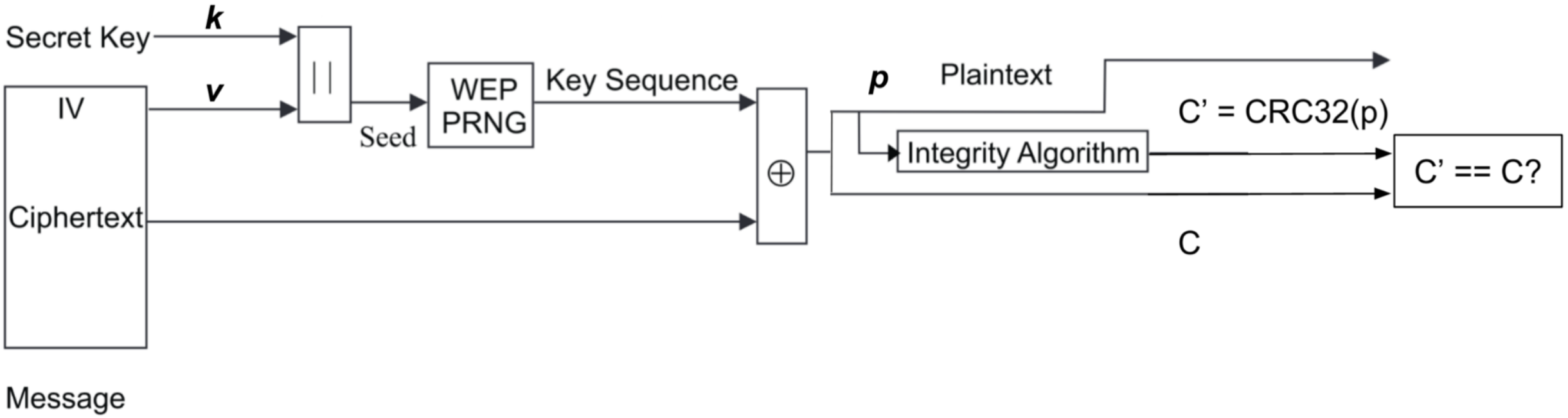
Q: How do we decrypt?

WEP Decryption



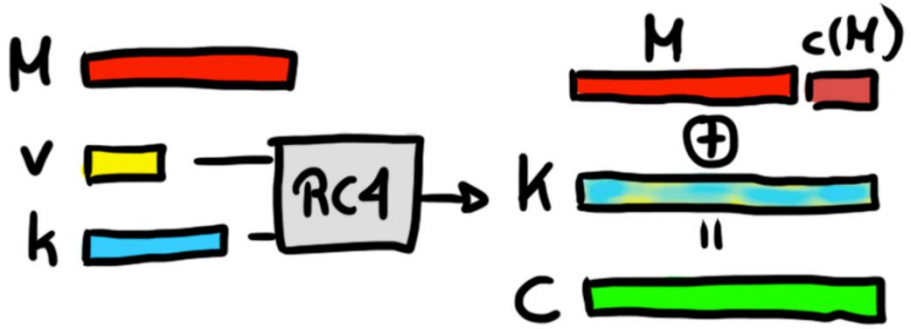
Here, C = CRC32(p)

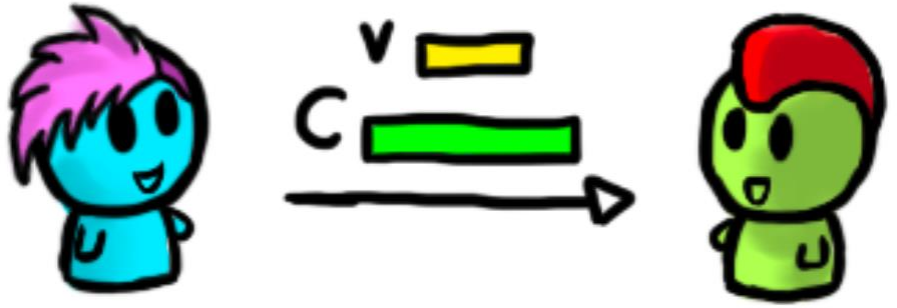
WEP Decryption



Looks… ok? What's the issue?

Problem 1: Key Reuse

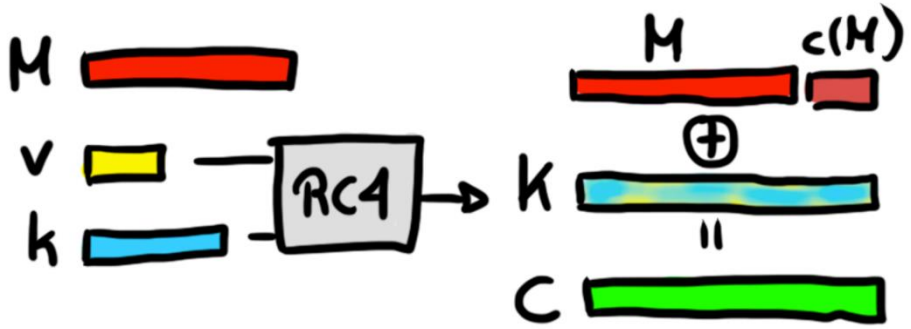


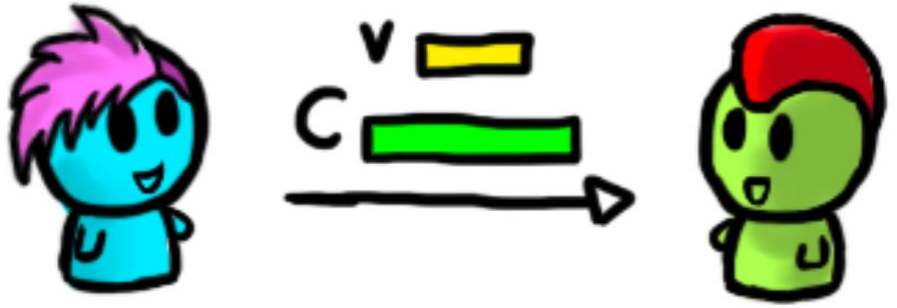


- IV (v) is too short: only 3 bytes = 24 bits.
- Secret (k) is rarely changed!

Q: What is the problem with this?

Problem 1: Key Reuse



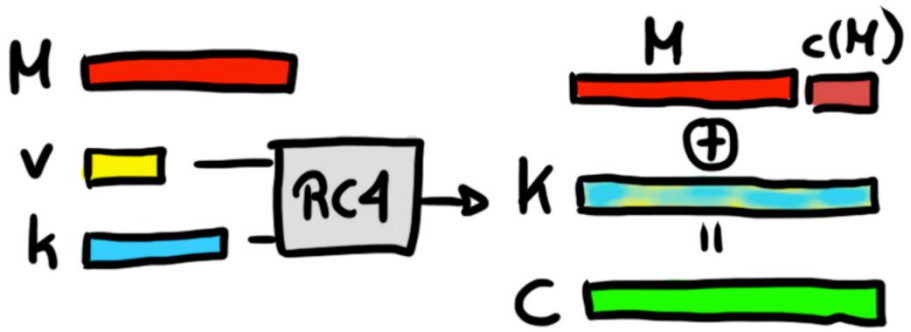


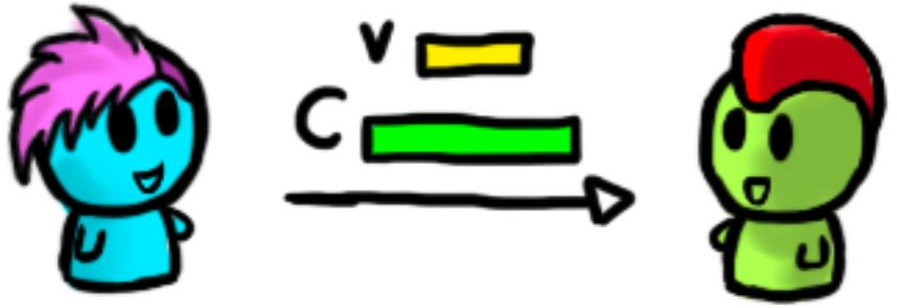
- IV (v) is too short: only 3 bytes = 24 bits.
- Secret (k) is rarely changed!

Q: What is the problem with this?

A: Key-stream gets re-used after (at most) 2^{24} protocol iterations (~17M packets) → two-time pad

Problem 2: Integrity?



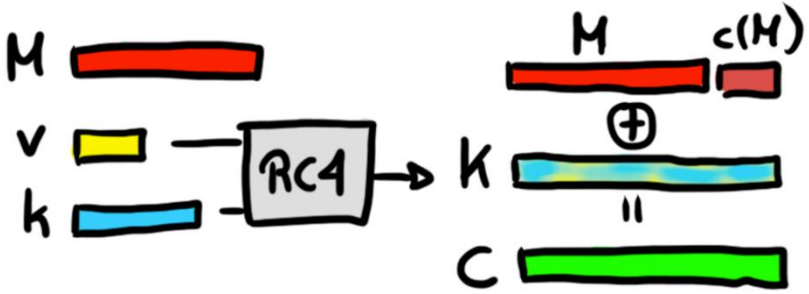


The checksum (*c*) algorithm in WEP is CRC32 → has two <u>undesirable</u> properties:

- It is independent of k and v
- It is linear: $c(M \oplus \delta) = c(M) \oplus c(\delta)$

Q: What is the problem with this?

Problem 2: Integrity?



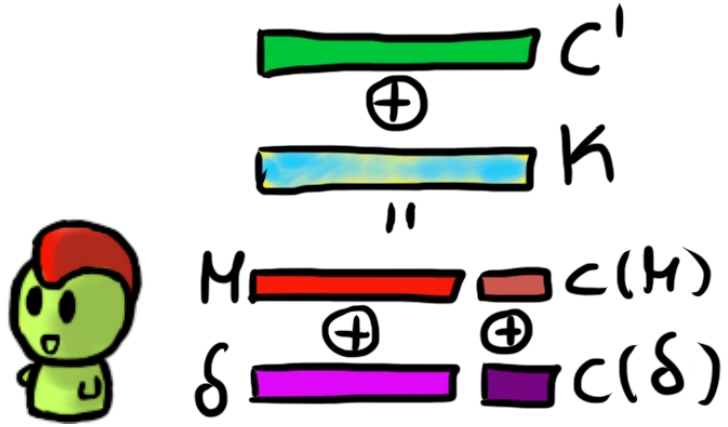


The sender transmits C and v. Mallory can modify the plaintext M into M' = M ⊕ δ:

- Calculate C' = ⟨M ‖ c(M)⟩ ⊕ ⟨δ ‖ c(δ)⟩
- Send (C',v) instead of (C,v)
- This passes the integrity check of the recipient!

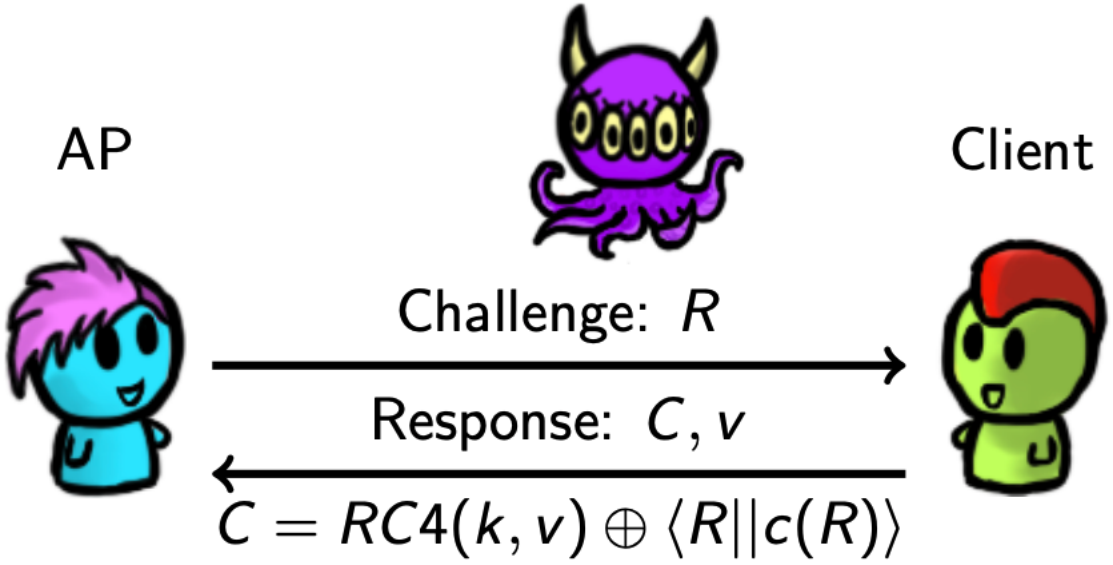
Q: How can we avoid this?

A: See below



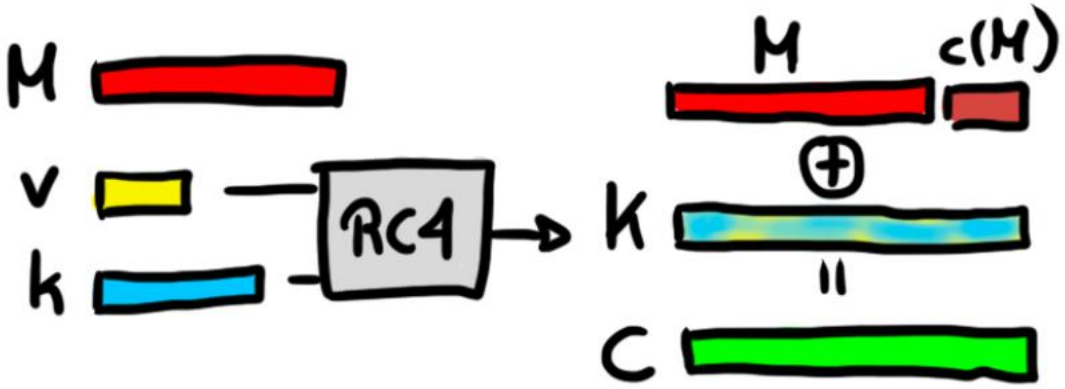
How does WEP authenticate?

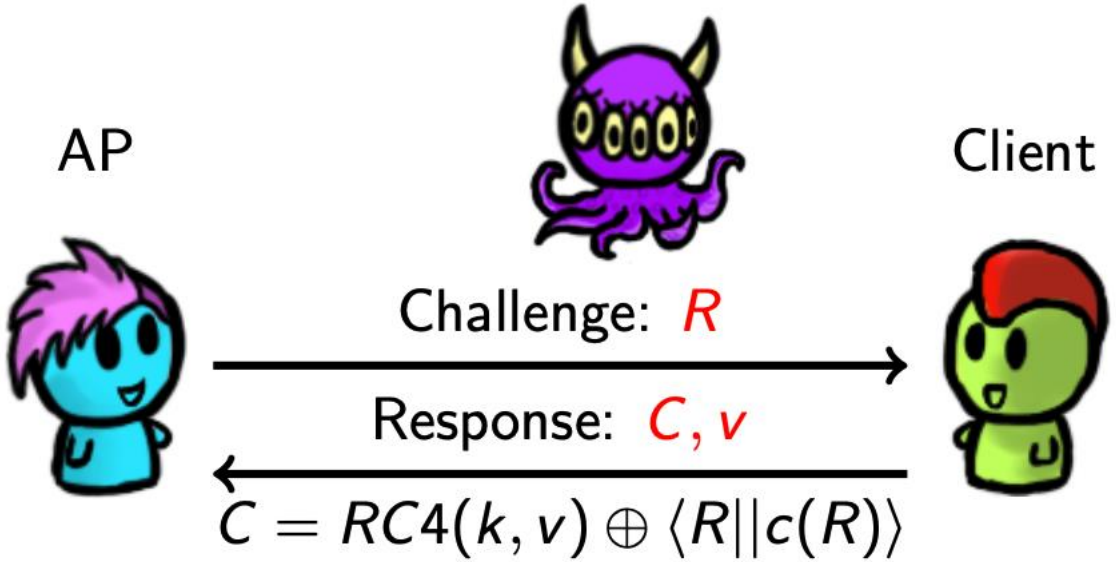
- **R** is a random challenge string
- Client encrypts **R** to prove knowledge of **k** to the AP
- If encrypted correctly, AP accepts client!



AP = Access Point, a.k.a. your wireless router

Let's break WEP authentication!

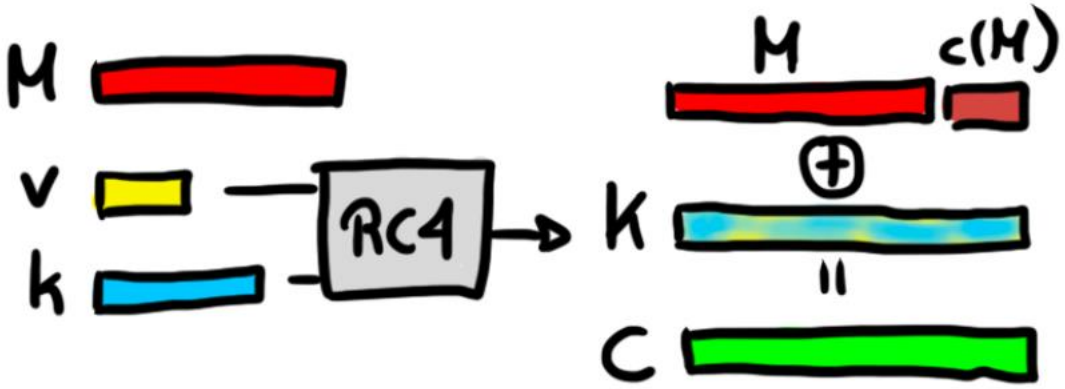


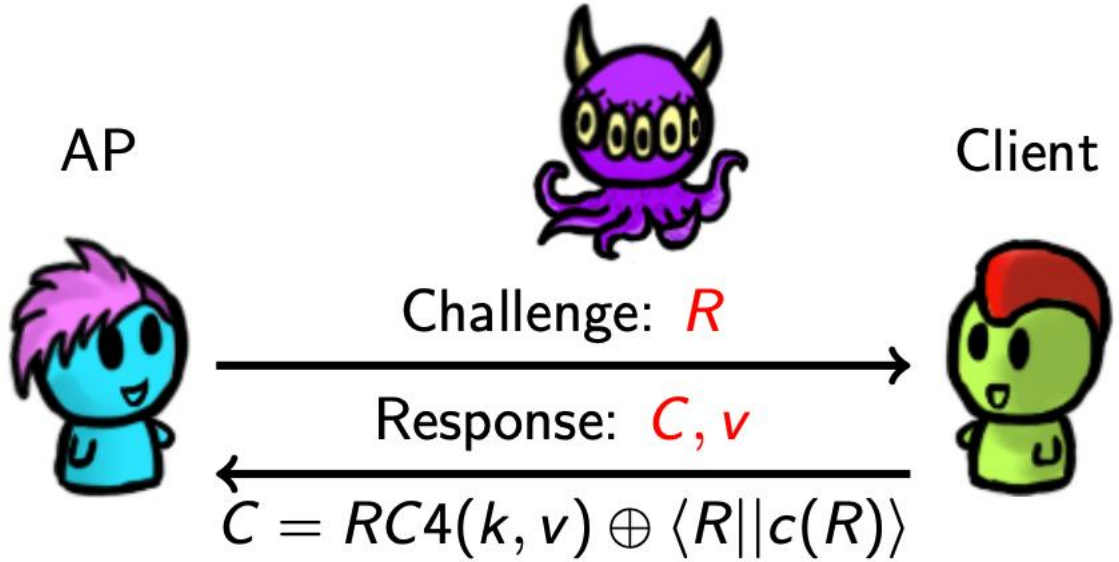


Mallory has seen R, C, and v.

Q: Mallory wants to authenticate herself to the AP. The AP sends Mallory a new challenge R'. Can Mallory successfully run the authentication protocol?

Let's break WEP authentication!





Mallory has seen R, C, and v.

Q: Mallory wants to authenticate herself to the AP. The AP sends Mallory a new challenge R'. Can Mallory successfully run the authentication protocol?

A: Yes! Note that Mallory can compute **RC4(k,v) = C ⊕ ⟨R||c(R)⟩** herself!!
Mallory can then compute: C' = RC4(k,v) ⊕ ⟨R'||c(R')⟩ and send C' and v to the AP.

Let’s break WEP authentication!

- How did the adversary get that single plaintext/ciphertext pair required for the attack on the previous slide?
  - It turns out the authentication protocol gives it to the adversary for **free**!
- This is a major disaster in the design!
- The authentication protocol is supposed to prove that a certain client knows the shared secret *k*
- But if I watch you prove it, I can execute the protocol myself!

Problem 3: Packet injection!?!

- We saw that observing **R**, **C**, and **v** gives Mallory a value of **v** and the corresponding keystream **RC4(v,k)**
- The same way Mallory encrypted the challenge R' in the previous slide, she can encrypt any other value **F**:
  - **C' =⟨F‖c(F)⟩ ⊕ RC4(v,k)**, and she transmits v,C'
- C' is a correct encryption of F, so the AP accepts the message

Problem 3: Packet injection!?!

- We saw that observing **R**, **C**, and **v** gives Mallory a value of **v** and the corresponding keystream **RC4(v,k)**
- The same way Mallory encrypted the challenge R' in the previous slide, she can encrypt any other value **F**:
  - **C' =⟨F‖c(F)⟩ ⊕ RC4(v,k)**, and she transmits v,C'
- C' is a correct encryption of F, so the AP accepts the message

So what?

Escalate



- Somewhat surprisingly, the ability to modify and inject packets leads to ways in which Mallory can trick the AP to **decrypt** packets!
  - Check out Prof. Goldberg's talk if you are interested.

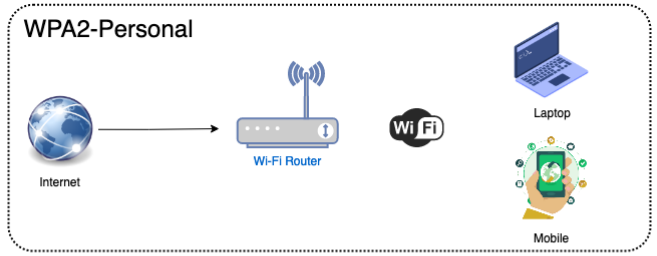
- None of the attacks so far use the fact that the stream cipher was RC4
  - When RC4 is used with similar keys, the output keystream has a subtle weakness
  - Leads to recovery of either a 104-bit or 40-bit WEP key in under 60 seconds
  - Check this paper for more details

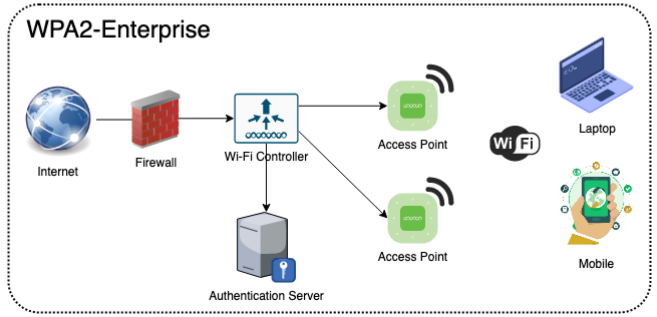
What have we learnt from WEP?

- Need to have sufficient randomness
  - Use **long keys** and **long IVs**.
  - Don’t reuse short-term secret keys and IVs.
- Do not use checksums for integrity.
  - Use **keyed MACs** instead! They are not linear.
- Go through public reviews of cryptographic protocols before standardizing them!
  - This helps **find weaknesses**.

Wi-Fi Protected Access II (WPA2)

- Has been required for products calling themselves “Wi-fi” since 2006
- Replaces RC4 with the CCM authenticated encryption mode (using AES)
- IV is 48 bit
- Replaces checksum with a real MAC
- Key is changed frequently
- Ability to use a 802.1x authentication server
  - But maintains a less-secure PSK (Pre-Shared Key) mode for home users
    - Allows for an offline dictionary attack…
    - Avoided in WPA3 (2018+)





Comparing Wi-Fi Protocols

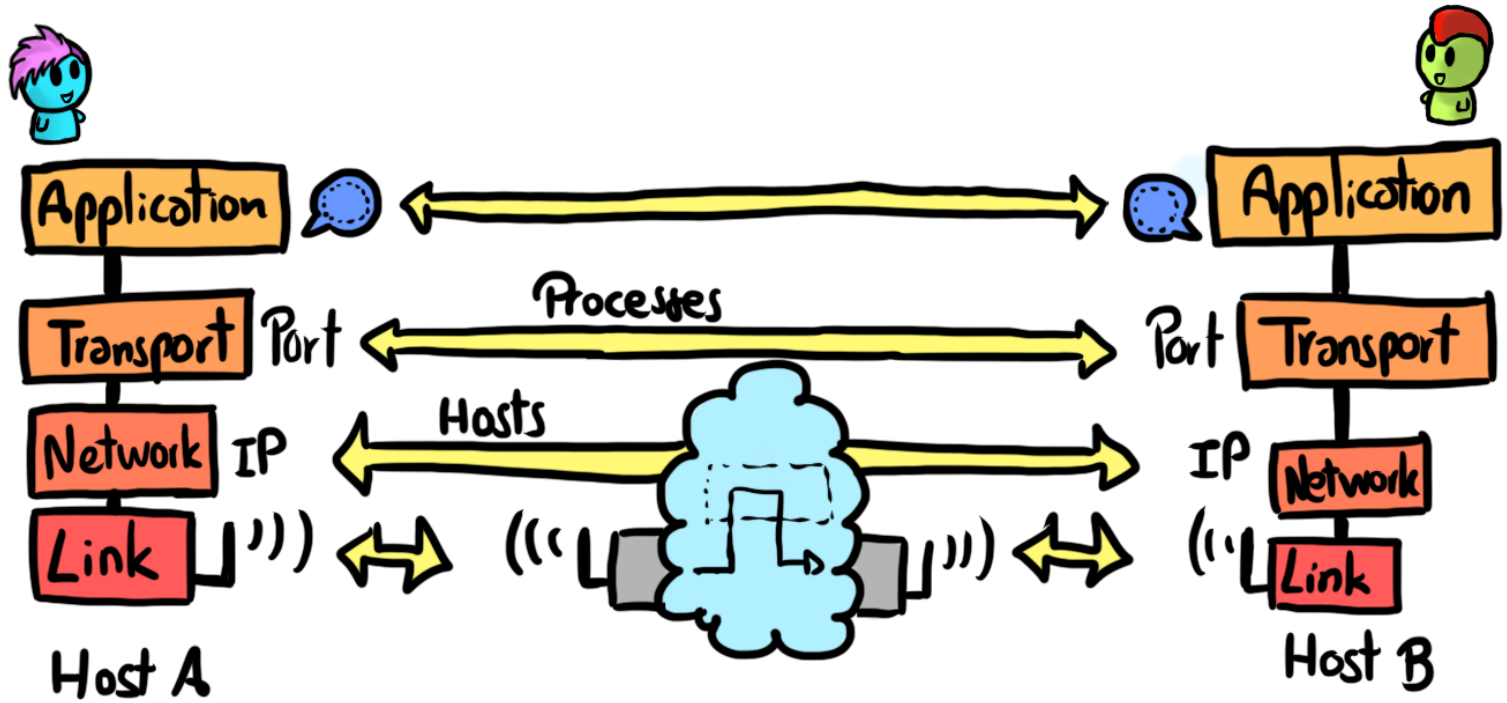
	WEP	WPA	WPA2	WPA3
Release Year	1999	2003	2004	2018
Encryption Method	Rivest Clipher 4 (RC4)	Temporal Key Integrity Protocol(TKIP) with RC4	CCMP and Advanced Encryption Standard	Advanced Encryption Standard(AES)
Session Key Size	40-bit	128-bit	128-bit	128-bit(WPA3-Personal) 192-bit(WPA3-Enterprise)
Clipher Type	Stream	Stream	Block	Block
Data Integrity	CRC-32	Message Integrity Code	CBC-MAC	Secure Hash Algorithm
Key Management	Not provided	4-way handshaking mechanism	4-way handshaking mechanism	Simultaneous Authentication of Equals handshark
Authentication	WPE-Open WPE-Shared	Pre-Shared Key(PSK)& 802.1x with EAP variant	Pre-Shared Key(PSK)& 802.1x with EAP variant	Simultaneous Authentication of Equals(SAE)&802.1x with EAP variant

Source: FS Community

Network Layer – VPNs

Security through the layers

- Link
  - WEP, WPA, WPA2
- Network
  - VPN, IPsec
- Transport
  - TLS/SSL
- Application
  - ssh (Next class: PGP, OTR, Signal)



Why do we need network layer security?

Suppose every link in our network had strong link-layer security. Why would this not be enough?

- Source & destination IPs may not share the same link.
  - Prone to network layer threats such as IP spoofing.
- We need end-to-end security across networks.

Virtual Private Networks

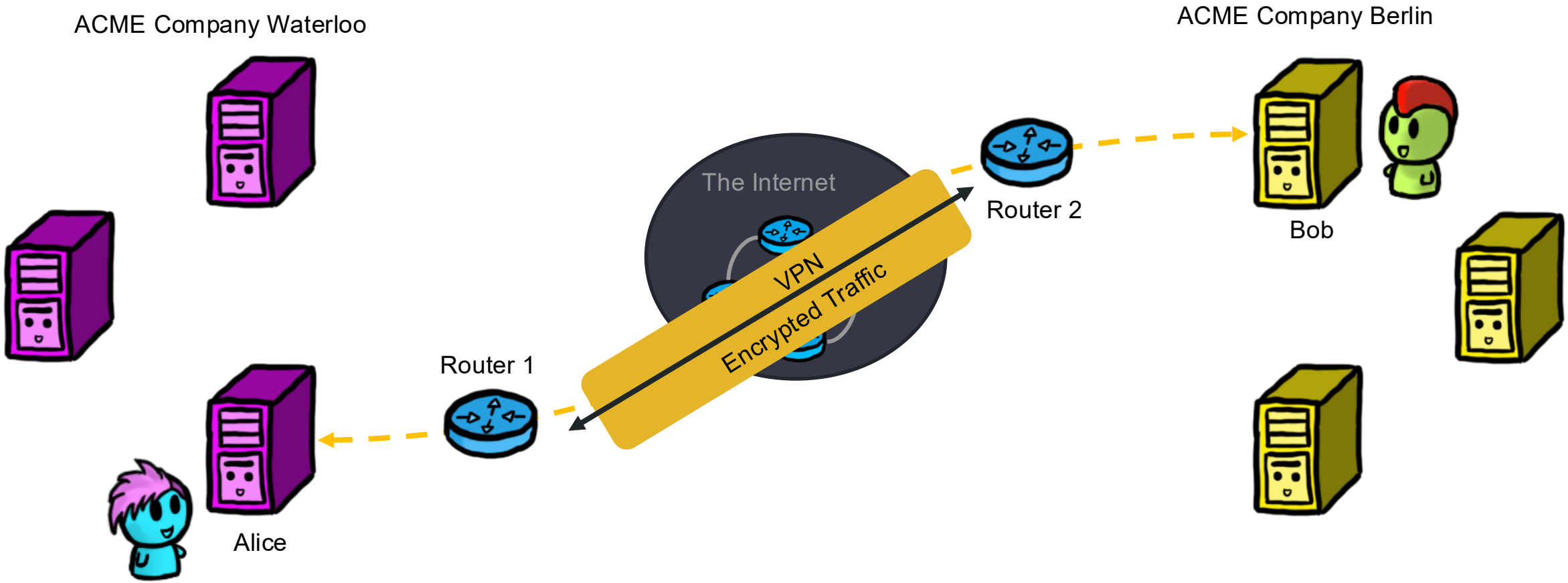


Private network: Has firewalls, access control and authentication so it is only used by trusted users.

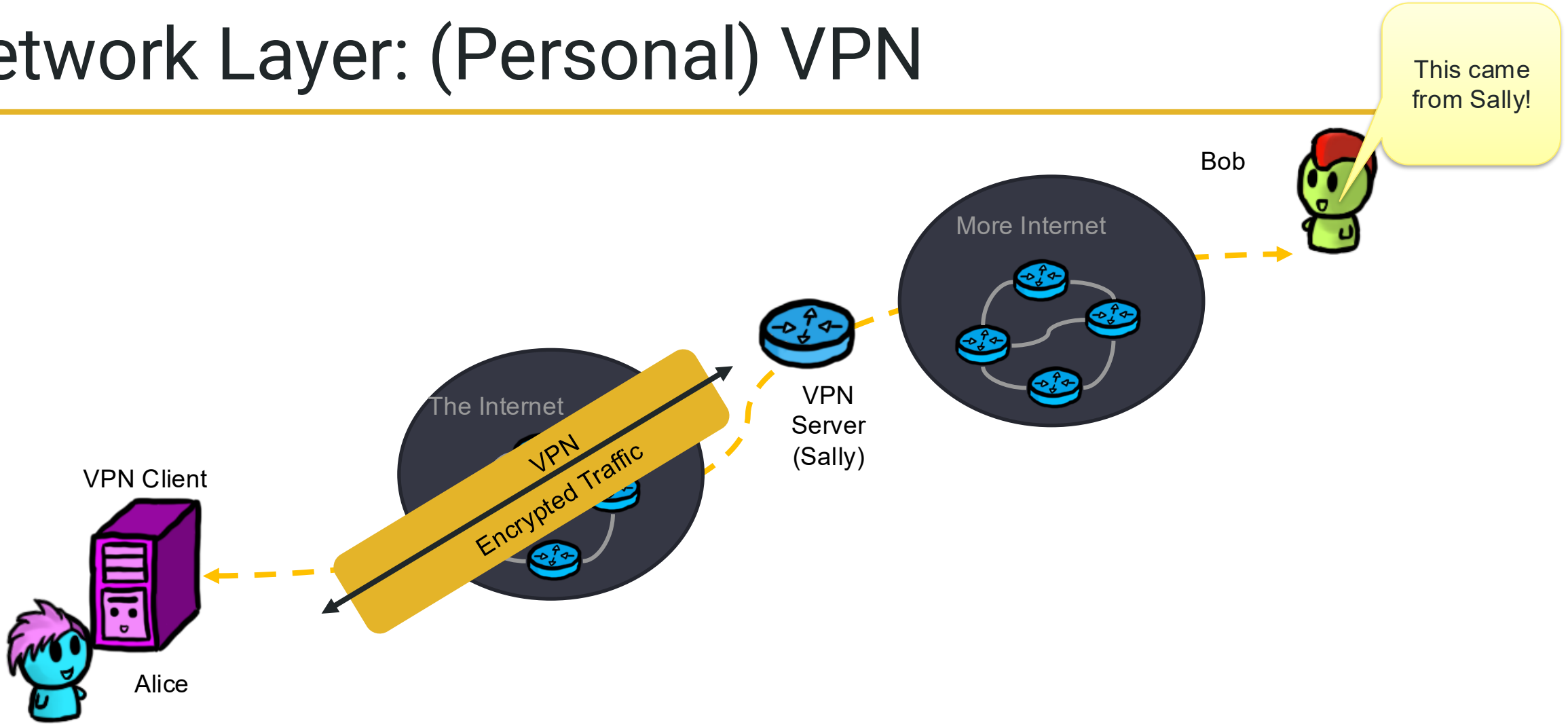


Virtual private network: A private network that connects physically distant users via virtual links, that are secured via cryptography

Network Layer: (Corporate) VPN



Network Layer: (Personal) VPN



“Interesting” Traffic

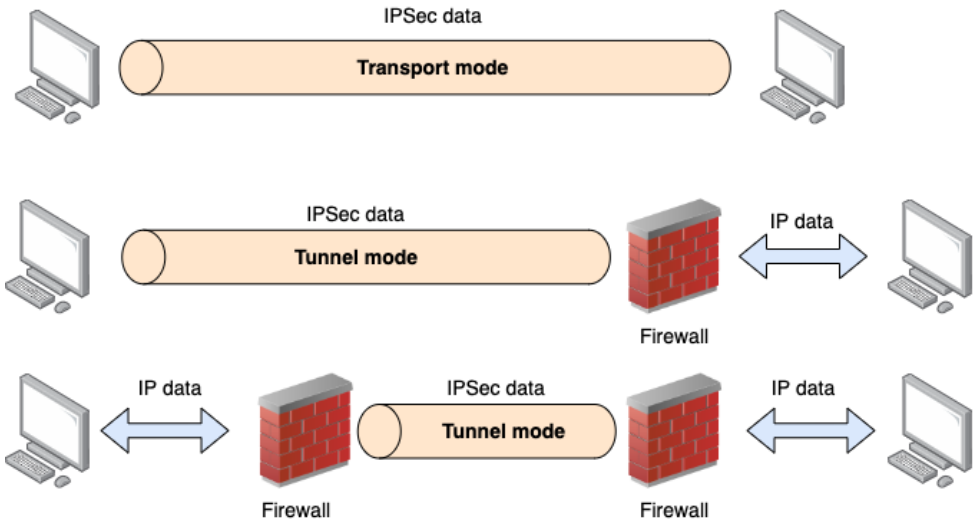
- In a corporate VPN, the VPN gateway can be configured to protect only “interesting traffic”
  - Furthermore, different types of traffic can go down different tunnels
- Similar to a firewall
  - Can be based on IP address, type of traffic, etc.
- Not usually the case in personal VPNs that protect everything.

IP Security suite (IPsec)

- Extends IP to provide confidentiality and integrity.
- It has two main modes:
  - Transport Mode
  - Tunnel Mode

Transport vs Tunnel Mode

- **Transport:** for point-to-point protection
  - Does not hide the IPs
  - Just encrypts the IP payload
  - Less common
- **Tunnel:** network-to-network or point-to-network
  - Hides the (inner) IP header and payload
  - Multiple variants:
    - Extend network across internet
    - Working remotely
    - Personal VPNs



Components of IPsec

1. Security Association (**SA**): Determine algorithms and keys
    - Decide MAC and encryption scheme (typically AES and SHA256), generate keys etc.
    - Uses IKE protocol.
2. Then, add either:

    A) Authentication Header (**AH**):
    - Provides integrity only

    B) Encapsulating Security Protocol (**ESP**)
    - Provides integrity and encryption

- How do we distinguish between the two IPsec formats?
    - “Protocol” field in the first IP header is set to 50 for AH, 51 for ESP

Authentication Header (AH)

- Offers integrity and data source authentication
  - Authenticates payload and parts of outer IP header that do not **usually** get modified during transfer
- Protects against replay attacks
  - Via extended sequence numbers

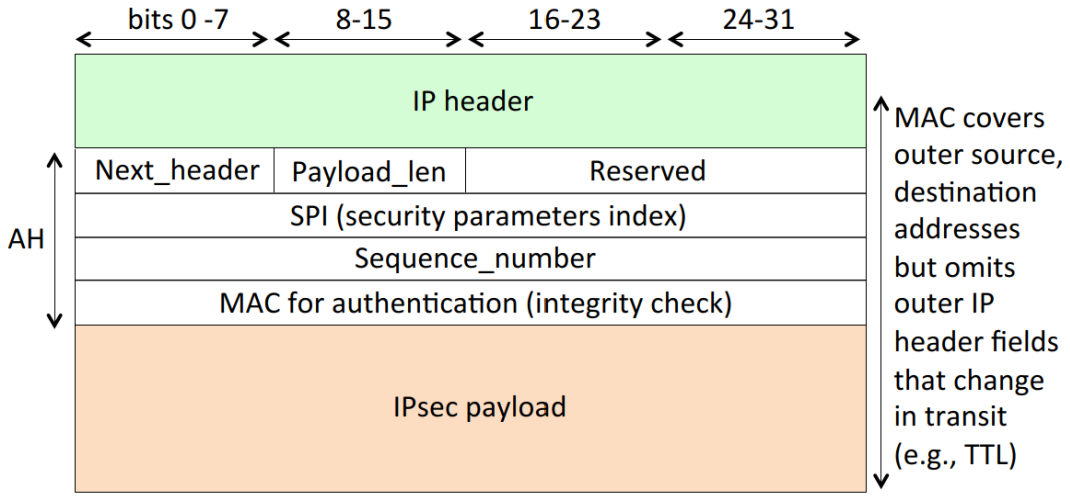
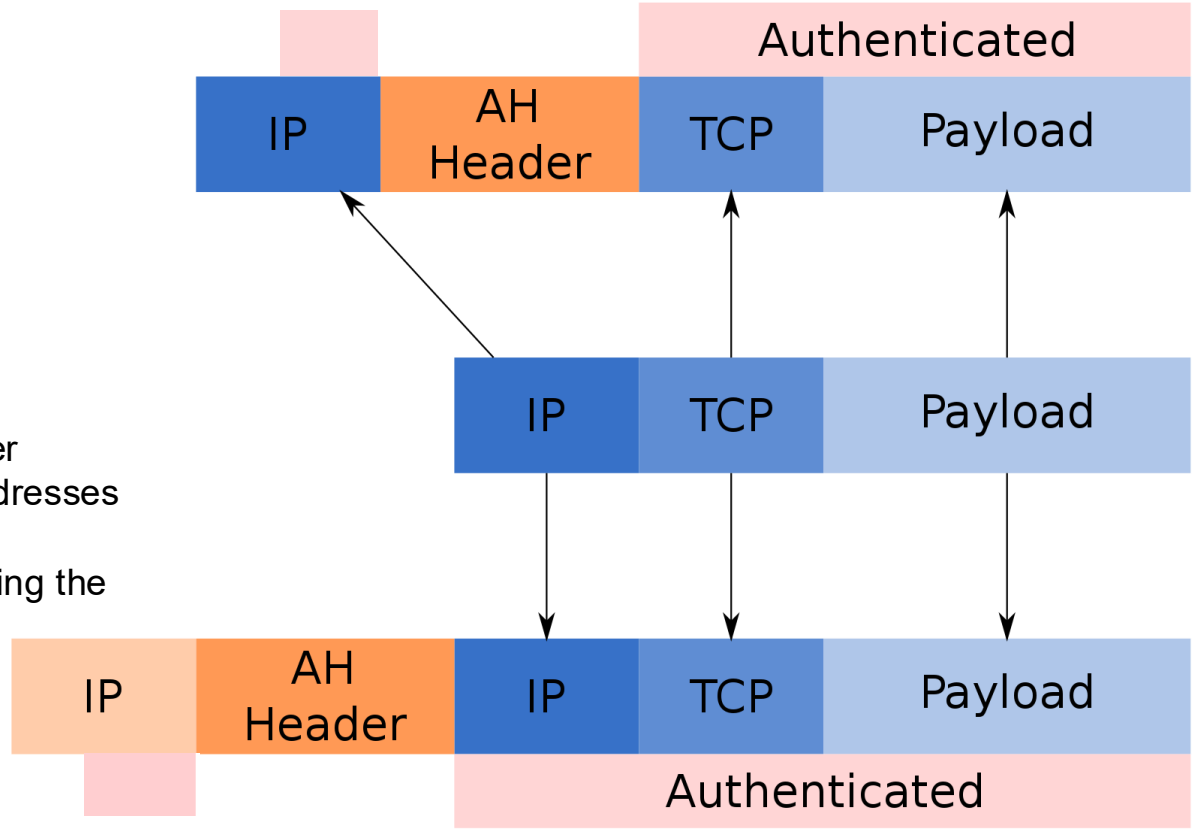


Figure 10.11: IPsec Authentication Header (AH) field view, for both transport and tunnel modes. `Next_header` identifies the protocol of the AH payload (e.g., TCP=6). `Payload_len` is used to calculate the length of the AH header. `SPI` identifies the Security Association. `Sequence_number` allows replay protection (if enabled).

Authentication Header (AH)



AH Transport Mode

Original Packet

AH Tunnel Mode

Outer IP header contains IP addresses of the two VPN gateways forming the tunnel

Encapsulated Security Payload (ESP)

- Offers confidentiality
  - IP data is encrypted during transmission
- Offers authentication functionality similar to AH
  - But authenticity checks only **focus on the IP payload**
- Applies padding and generates dummy traffic
  - Makes traffic analysis harder

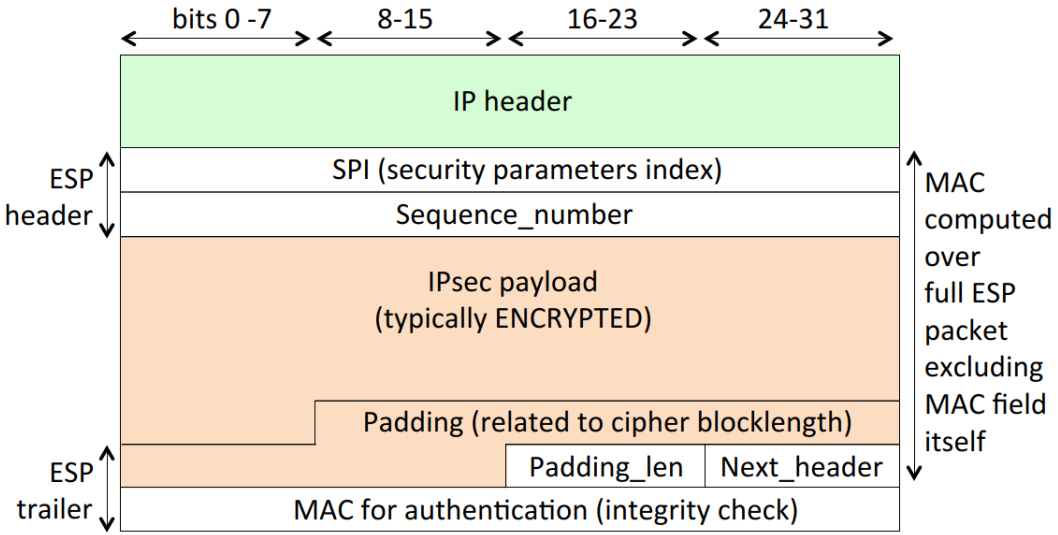
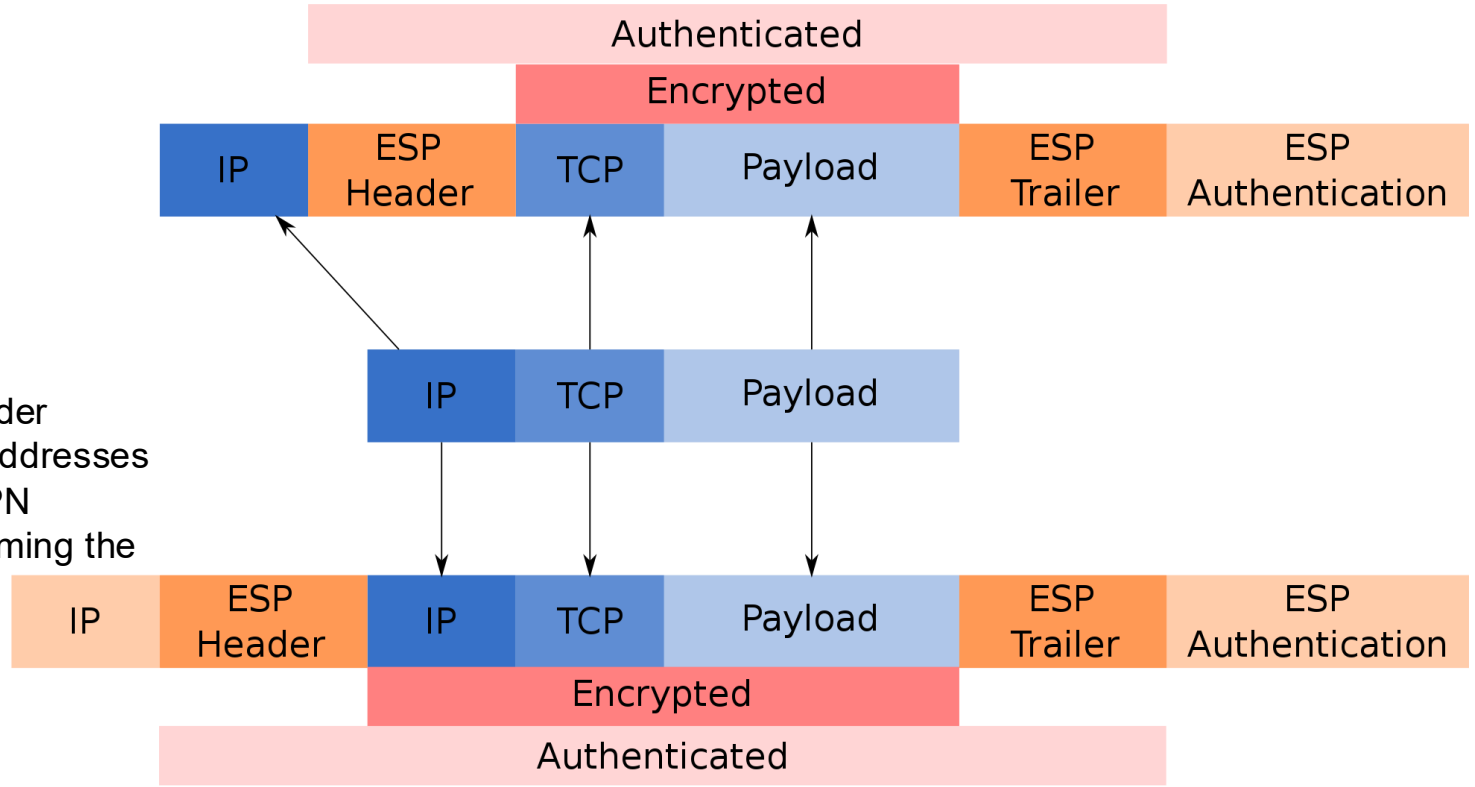


Figure 10.12: IPsec Encapsulating Security Payload (ESP) field view, for both transport and tunnel modes. `SPI` identifies the Security Association. `Sequence_number` allows replay protection (if enabled). `Next_header` (which may include a crypto IV or Initialization Vector) indicates the type of data in the ENCRYPTED field. A payload length field is not needed, as the ESP header is fixed at two 32-bit words, and the length of the IPsec payload (which is the same as that of the original payload) is specified in the IP header.

Encapsulated Security Payload (ESP)



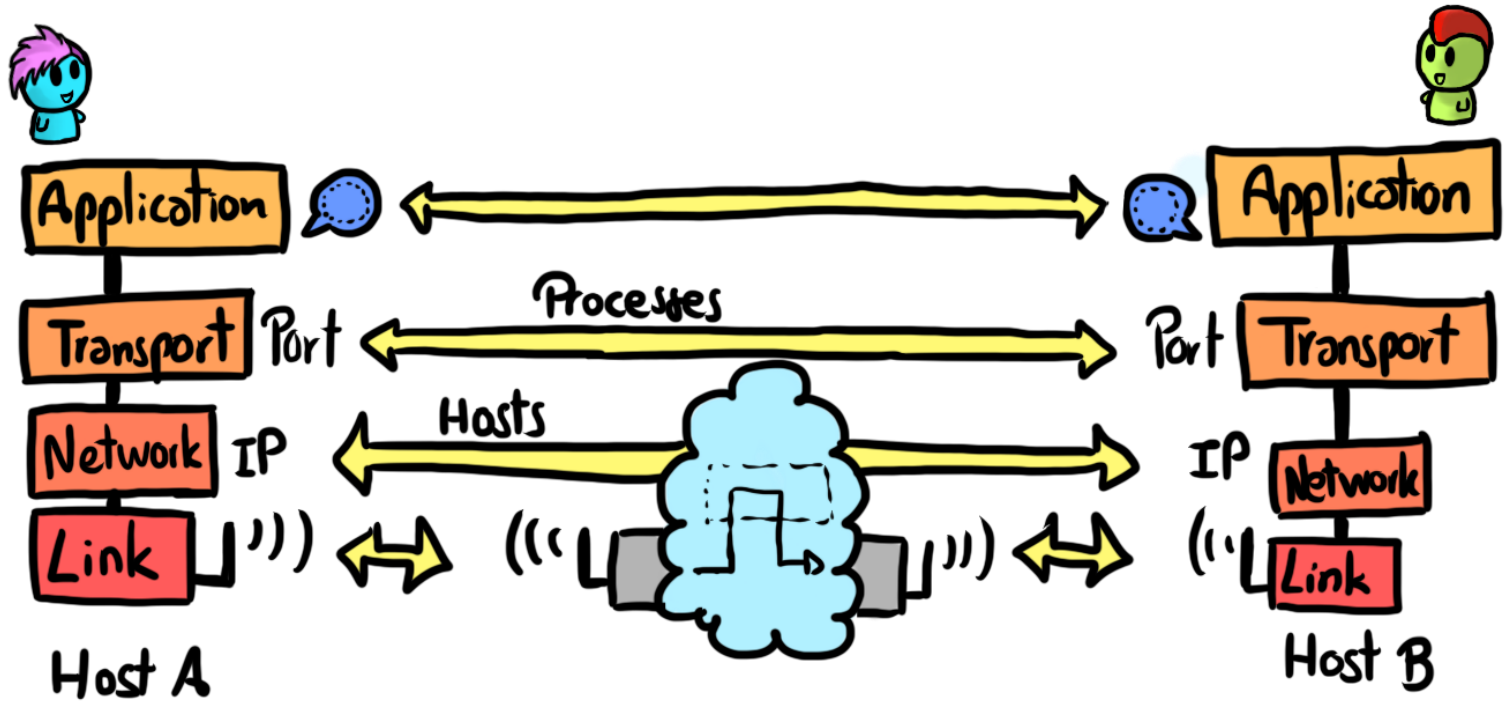
IPsec Deployment Challenges

- Needs to be included in the kernel's network stack.
- There may be legitimate reasons to modify some IP header fields; IPsec breaks networking functionalities that require such changes.
  - E.g., with AH, you cannot replace a private address for a public one at a NAT box.
  - The NAT cannot recalculate the authentication hash to match the changed IP address
- IPsec is complex, hard to audit, and prone to misconfigurations

Transport Layer: TLS

Security through the layers

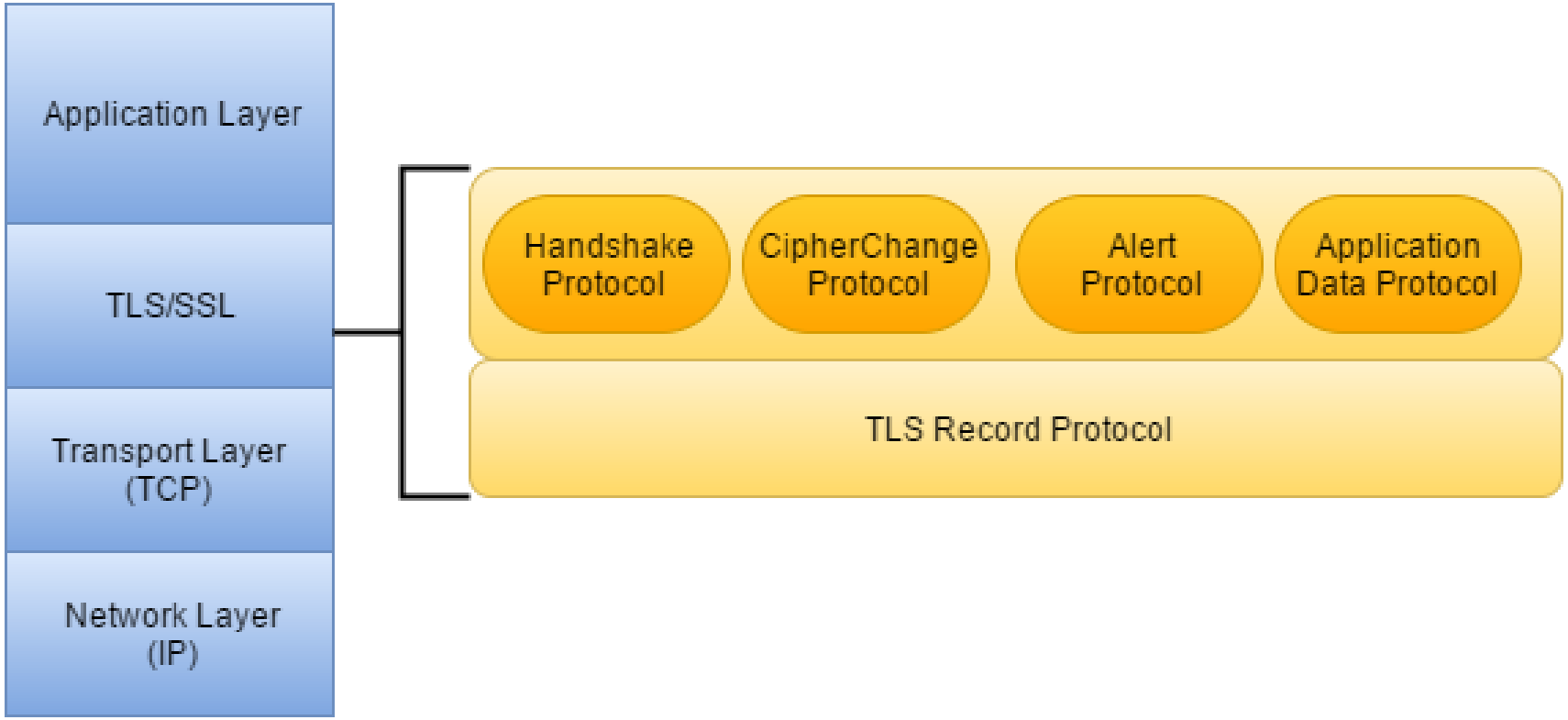
- Link
  - WEP, WPA, WPA2
- Network
  - VPN, IPsec
- Transport
  - TLS/SSL
- Application
  - ssh (Next class: PGP, OTR, Signal)



Transport Layer Security Purpose

- Closer to end-to-end security: Client to server
- Defense-in-depth when used in conjunction with IPsec.
  - **Network-layer** security mechanisms arrange to send individual IP packets securely from one network to another
  - **Transport-layer** security mechanisms transform TCP connections to add security and privacy

Where does TLS sit on the network stack?



Source: Vidhatha Vivekananda

TLS Record Protocol

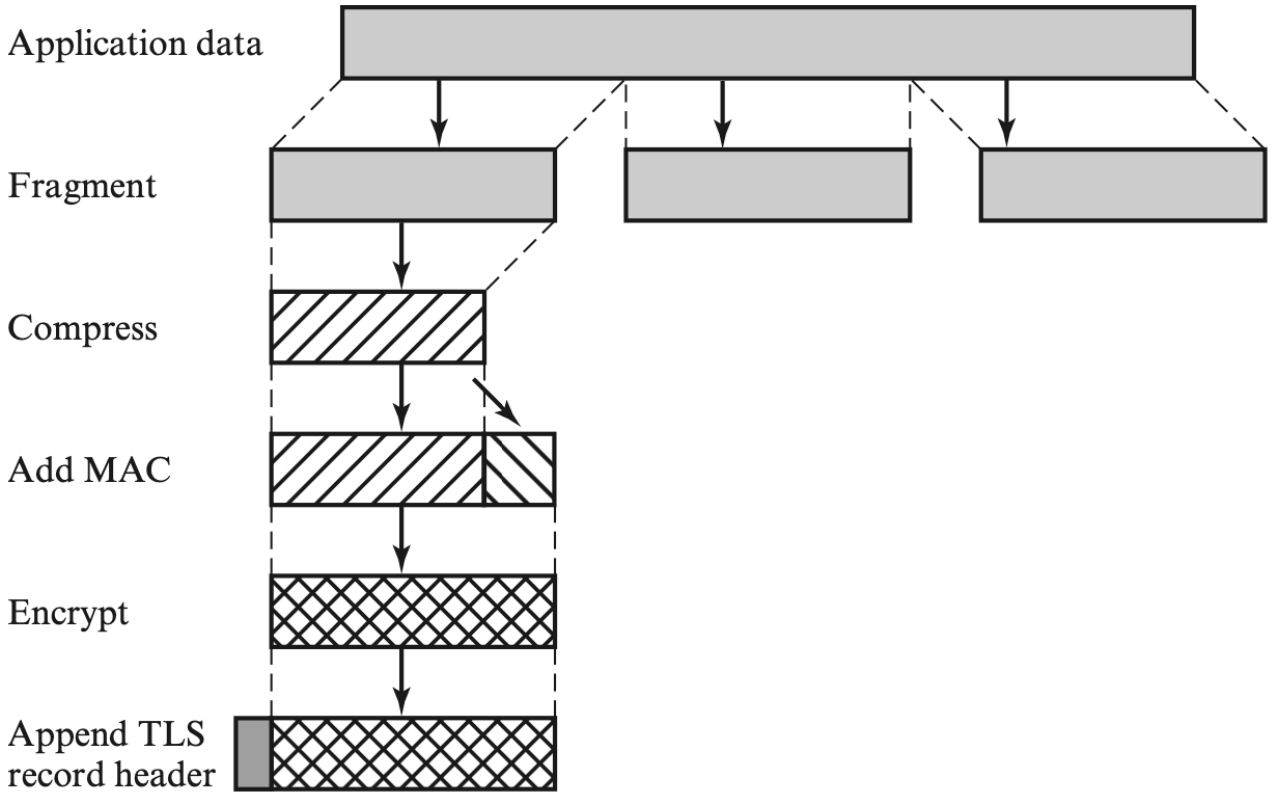
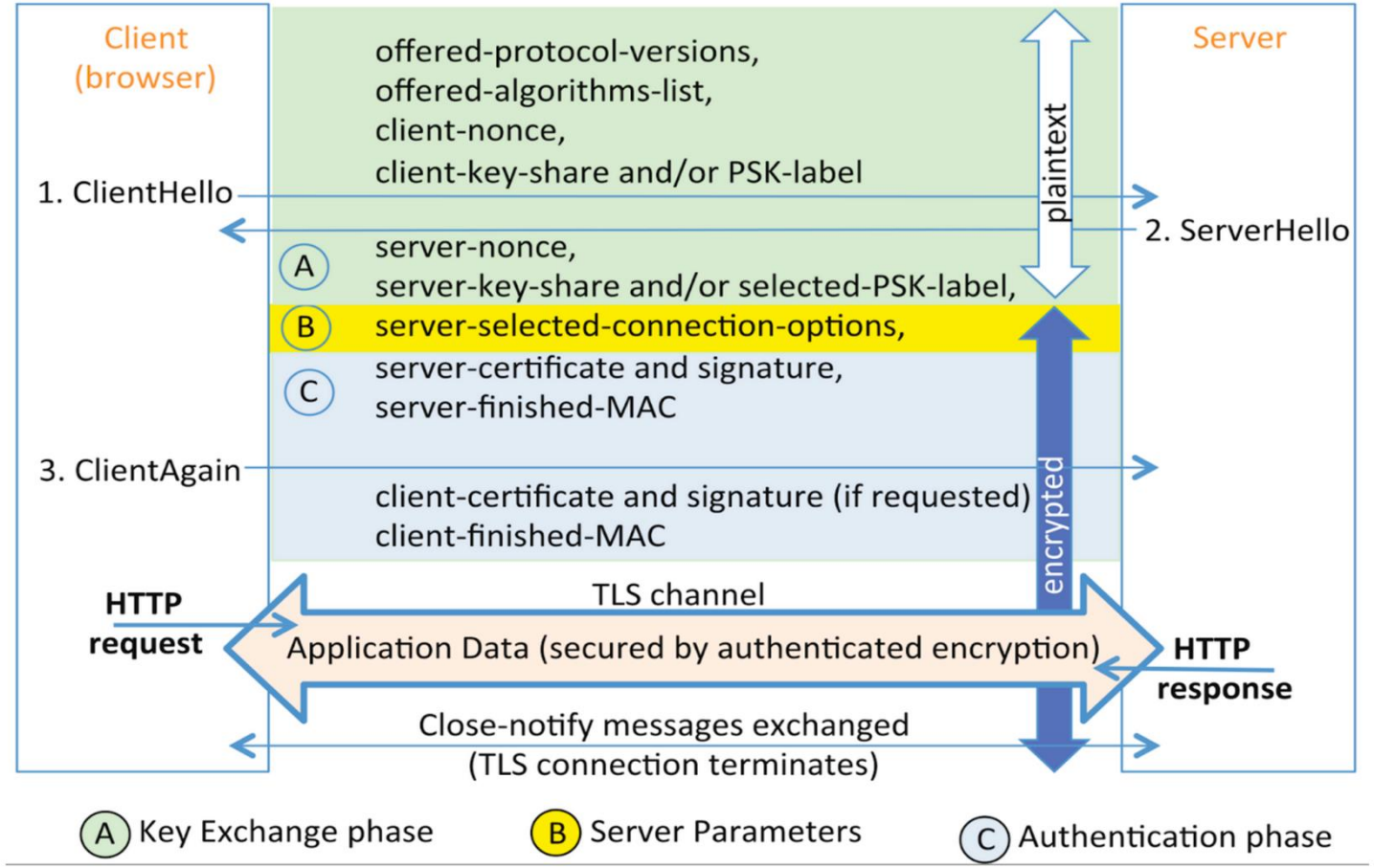


Figure 6.3 TLS Record Protocol Operation

The TLS Handshake – To Establish Sessions

- Client and server agree on TLS version and cipher suites to be used
- Client and server run a key-exchange protocol (like DH)
- Client authenticates the identity of the server
  - Hostname of server (like from URL) must match hostname in certificate received from server
  - Certificate must be signed by a CA that client trusts (likely multiple levels of CAs and certificates are involved)
  - Server must sign all handshake messages with signing key corresponding to verification key in certificate
- Derive two session keys from established key to use symmetric encryption and MACs after the handshake is complete

TLS 1.3 Handshake



TLS Design Choices

- Highly configurable protocol with many options/versions
  - Different authentication/key exchange protocols
  - Different encryption and signature algorithms
- Authentication
  - Usually (!) one-sided, only server authenticates
  - Server PKI certificates
  - Secure Channel: Software distribution
- Hybrid encryption (symmetric for data, asymmetric for key exchange)
  - Key Exchange: Authenticated Diffie-Hellman

CAs in TLS

A certification authority acts as a trusted third-party that:

- Issues digital certificates
- Certifies the ownership of a public key by the subject of the certificate
- Manages certificate revocation lists (CRLs)

Why one-sided authentication?

- PKI is a “somewhat” closed system
  - Difficult to obtain certificate
  - Difficult to manage keys
  - User-unfriendly
- Web traffic contains “few” servers and many clients
  - Efficient way of implementing authentication

Preventing Modifications by Mallory

- Authenticated Encryption
  - MACs with every “packet”
  - Mallory cannot modify packet
- Can Mallory drop a packet?
  - MAC is dropped alongside with it
  - Nothing to verify
- Can Mallory replay a packet?
  - MAC is correct
  - Solution: **Sequence Numbers**

Doesn’t TLS suffice?

Or, why do I need Link or Network layer protection anymore?

- TLS only encrypts the payload, not source/destination IP.
- Still don’t want to expose internal network via Wi-Fi.
- Redundancy!

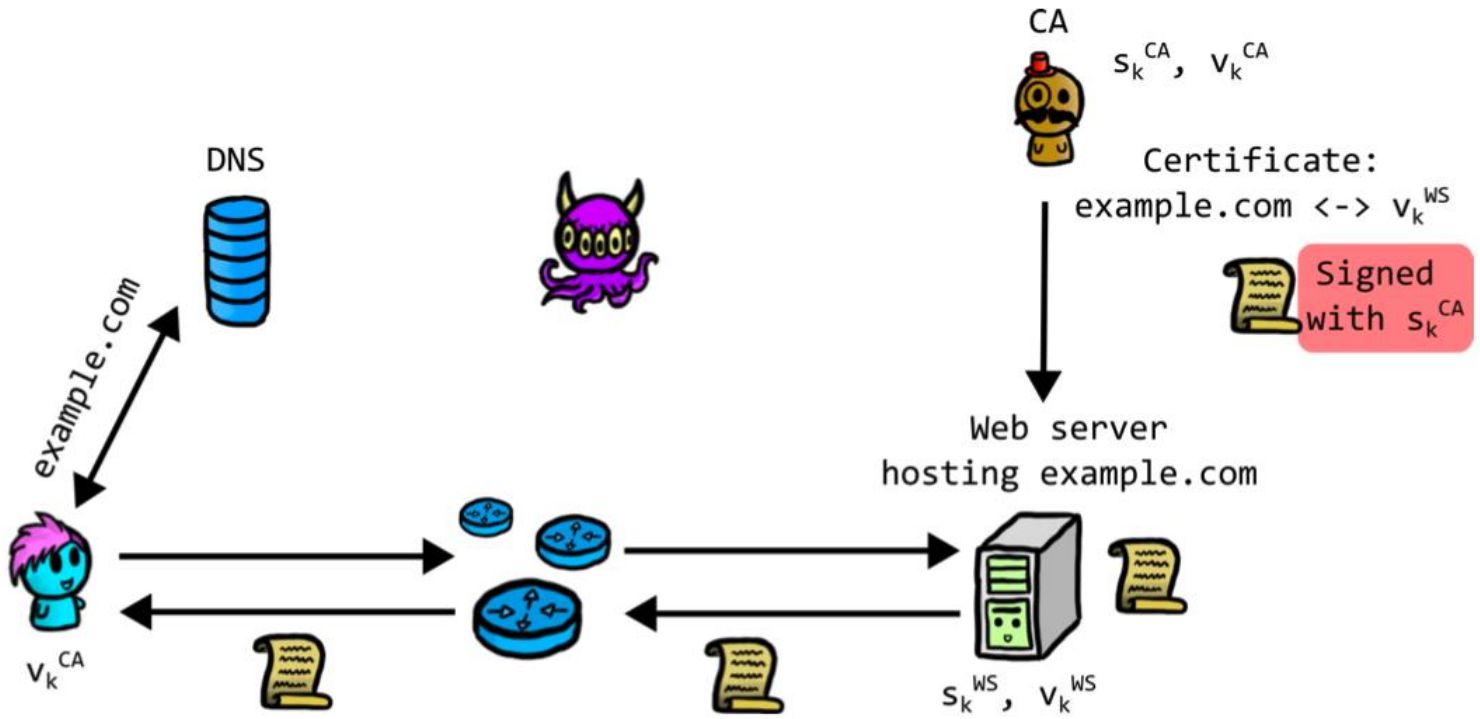
What can go wrong with TLS

- Implementation issues
- Using weak ciphers
- **Compromising CAs**

Recall TLS Authentication

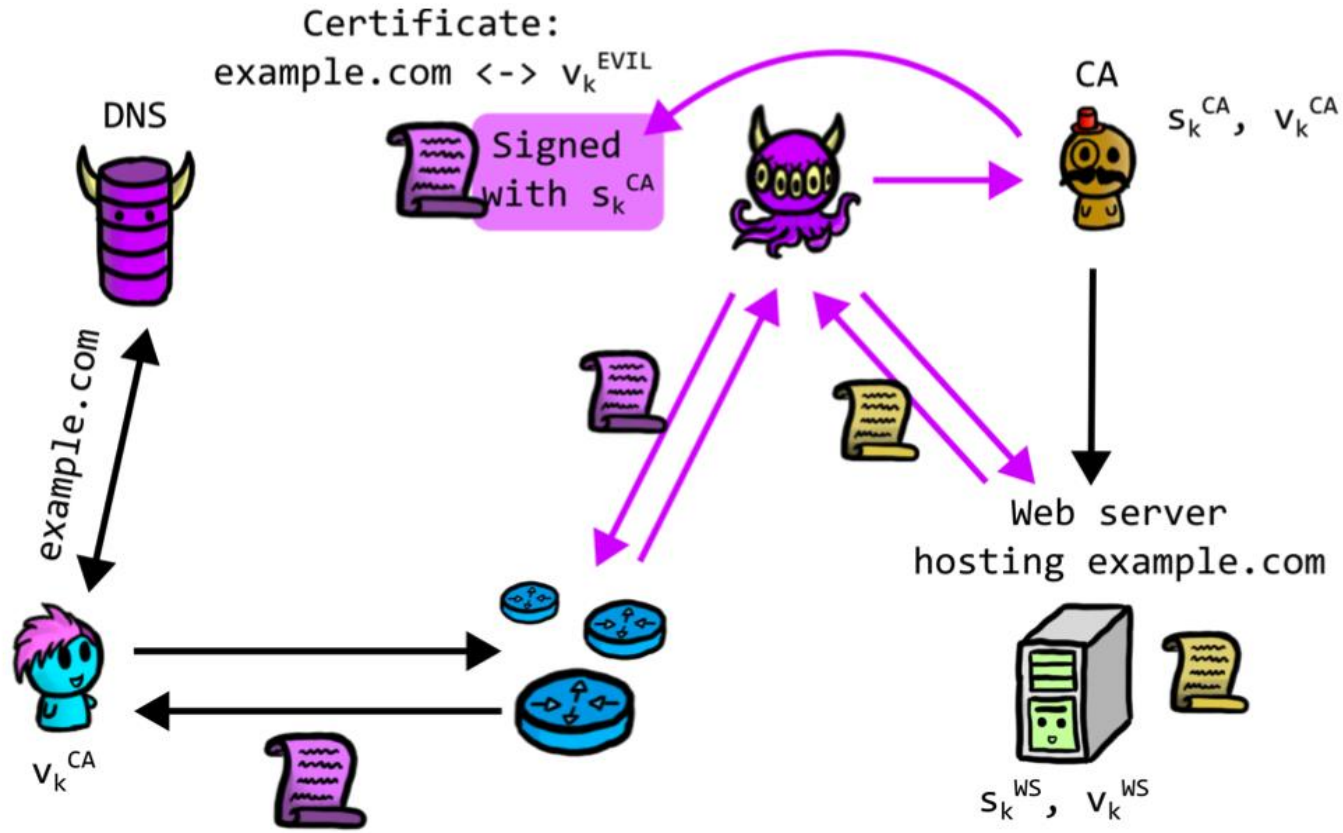
Basic idea: Alice accepts the connection if she receives a certificate and

1. the certificate is signed by a CA she trusts (v_k^{CA})
2. the certificate is for the domain she's requesting
3. when talking to the web server, Alice can verify the signatures with v_k^{WS} (which is in the certificate).



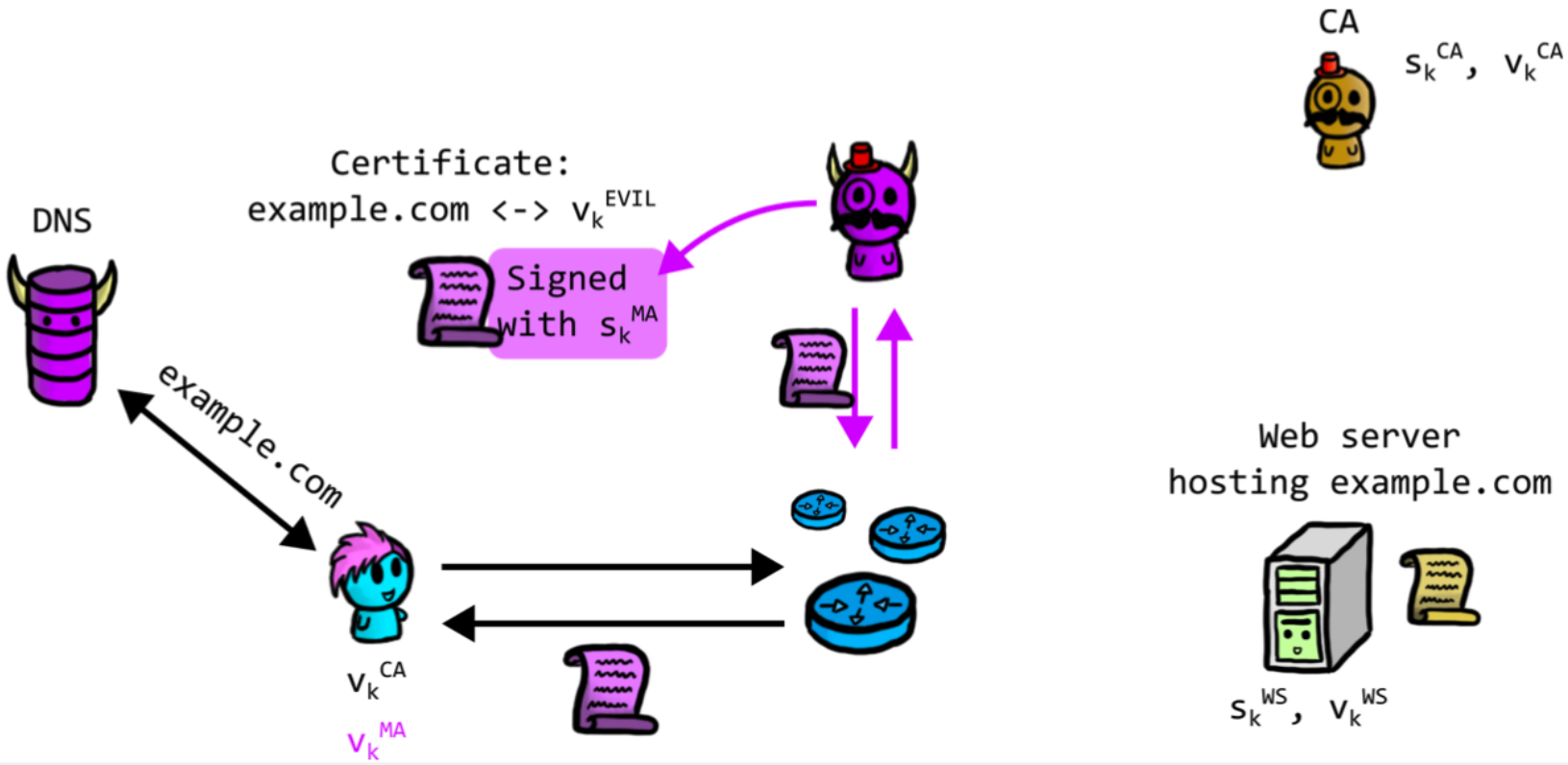
Compromising CAs – Planting Fake Certificates

An adversary can compromise a CA to plant fake certificates (e.g., DigiNotar's fake *.google.com certificates used by an ISP in Iran)



Compromising CAs – Total Bypass

An adversary can install a custom CA on users' devices, allowing them to sign certificates that clients will accept for any site (e.g., in 2019, Kazakhstan's ISPs mandated the installation of a root certificate issued by the government)



Compromising CAs – What? Why?

Companies may think it is an excellent idea

- Lenovo's Superfish
- Sennheiser HeadSetup



- In addition to installing a malicious CA verification key, these would also install the (same!) corresponding signing key on all affected machines
- Allows to dynamically issue certificates and break up encrypted connections

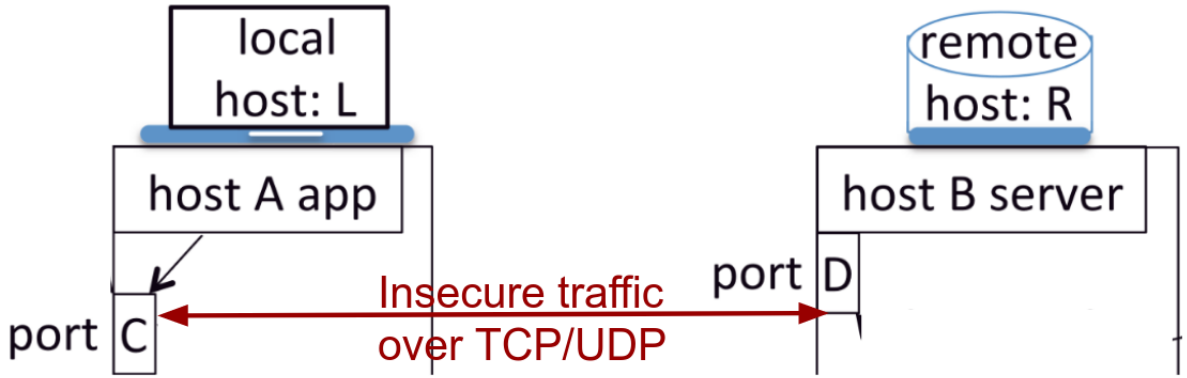


- For advertisement and communication purposes

Application Layer - SSH

Pre-SSH

- Suppose that you want to connect to a remote machine
  - You may think “Oh ok, let me use Telnet”
- Think again...
  - All data exchanged through Telnet is in plain text!



Enter Secure Remote Login - SSH

Usage (simplified):



- Client connects to server
- Server sends its verification key
  - The client should verify that this is the correct key
  - Many clients implement trust on first use (**TOFU**)
- Client and server run a key agreement protocol to establish session keys, server signs its messages
  - All communication from here on in is encrypted and MAC-ed with the session keys
- Client authenticates to server
- Server accepts authentication, login proceeds

How does the client authenticate

There are two main ways to authenticate with ssh:

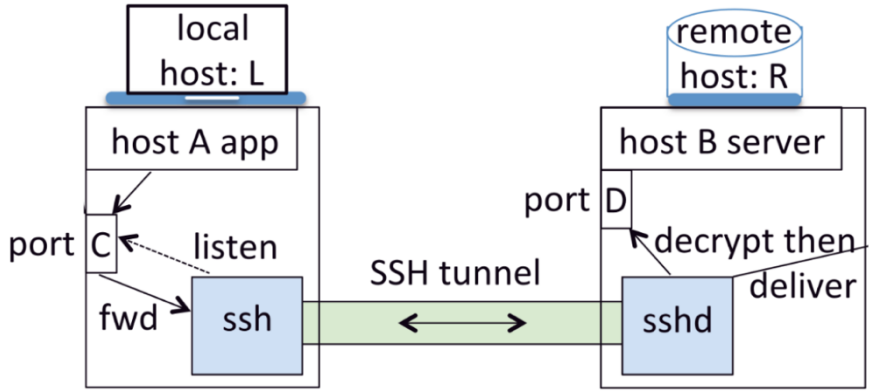
- Send a password over the encrypted channel
  - The server needs to compute (a salted hash of) your password
- Sign a random challenge with your private signature key
  - The server needs to know your public verification key

Q: Advantages / Disadvantages of each

SSH Port Forwarding

SSH allows for tunneling:

- The client machine can create a mapping between a local TCP port and a port in the remote machine
  - e.g., localhost:IMAP to mail.myorg.ca:IMAP
- The client SSH and the server SSH daemon operate as a secure relay
  - Allows the client to interact with server applications via SSH



Next Class: PGP, OTR, Signal